METR4202 Robotics & Automation
Semester 2, 2026
The University of Queensland

Team Project Interim Assessment Report

Autonomous Frontier Exploration and ArUco Marker Mapping with the TurtleBot3 Waffle Pi

Team 10

Team member	Student number	Primary area of contribution
Mitch Bradshaw	s46991580	Mapping packages
Brad Cronin	45801459	Mapping packages
Einar Augestad	s50492543	Exploration
Ingrid Skålnes Strømme	s50492534	Exploration

Submission date	26 October 2026
Prac session	Tuesday 9 AM, 78-209
Demonstration date	27 October 2026
Code repository	https://github.com/mitchbradshaw/metr4202_2026_team10

Table of Contents

Table of Contents **2**
Executive Summary **3**
Introduction **3**
Project Brief and Task 3
Platform and Software Environment 3
Report Structure 3
System Overview **4**
System Architecture Diagram 4
Packages and Nodes 4
Data Flow and Topics 6
Launch Sequence and Runtime Configuration 7
Developed Components **7**
Frontier Detection 7
Frontier Clustering and Waypoint Candidates 7
Information Gain Estimate 8
Driving Distance 8
Waypoint Scoring and Exploration Strategy 8
Marker Visiting 8
Explorer Node and Nav2 Goal Handling 8
Failure Detection and Recovery 9
ArUco Marker Detection 9
Marker Localisation in the Map Frame 9
Custom Message Interfaces 9
System Bringup 9
Test Worlds 10
Use of Existing Code and Libraries **10**
Interim Assessment Checklist Compliance **12**
Testing and Results **13**
Test Environments 13
Unit Tests 14
Simulation Results 14
Real Robot Results 14
Observed Failures and Limitations 15
Individual Contributions **15**
Installation and Operation **17**
Future Work Toward the Final Demonstration **17**

References **17**
Appendices **18**
Appendix A: Repository Structure 18
Appendix B: Tunable Parameters 19
Appendix C: Launch Arguments 21

Executive Summary

This project performs autonomous exploration of an unknown arena, mapping the arena using LiDAR SLAM, and using ArUco target detection to recognise and map ArUco markers within the arena. The robot explores by driving to frontier cells between mapped and unmapped space, scored by the unknown area visible from each cell against the driving distance to it, and visits each mapped marker once at close range. This submission showcases the current exploration functionality and marker detection capabilities in simulation and on the real robot. Since the previous version of this report, scoring uses driving distance and a ray-cast gain, markers are visited as goals, the robot scans left and right at each goal, and all tuning values are in a YAML configuration. The final submission will need further optimisation of exploration and marker placement, detailed in Future Work.

Introduction

Project Brief and Task

The brief is given as *'Develop an exploration strategy for your TurtleBot3 to search an unknown environment and locate targets in a map.'* The success criteria for the team is based on the ability for the robot to:

- Move without collision with walls or the environment
- Map markers to a map location within 0.5 m
- Complete an entire mapping of an environment in under 5 minutes

Platform and Software Environment

The physical hardware used for the demonstration is a TurtleBot3 Waffle Pi with LiDAR and front camera. The software environment involves the following requirements:

- Ubuntu 22.04 operating system
- ROS 2 Humble
- Gazebo Classic 11
- Nav2
- slam_toolbox

Report Structure

The following guide will detail the system in **System Overview**, the components of the algorithms in **Developed Components** and finally some results from testing in **Testing and Results**, with contributions named in **Individual Contributions** and future work detailed in **Future Work Toward the Final Demonstration**.

System Overview

System Architecture Diagram

The current system architecture is shown in Figure 1. It can be seen that there are 3 significant custom nodes in the architecture: camera_node, map_markers_node and frontier_explorer. The node camera_node subscribes to the camera data on board the TurtleBot3, detects the presence of ArUco markers and publishes detections to /aruco/detections. These publications are subscribed to by map_markers_node, which also subscribes to the slam_toolbox /tf node, to locate the relative position of the camera within the map at the time of detection. It uses this information to publish a marker location to RViz and to the explorer. The autonomous exploration of the robot is handled entirely by frontier_explorer, which subscribes to the current map estimate from slam, the log of robot behaviours and the overlaid costmap from Nav2, and the mapped markers, to detect the locations of unknown cells and adjacent known free 'frontier' cells, and act accordingly. All three custom nodes read their tuning values from the YAML files in team10_bringup/config. The algorithms for exploration are described in more detail later in this

report.

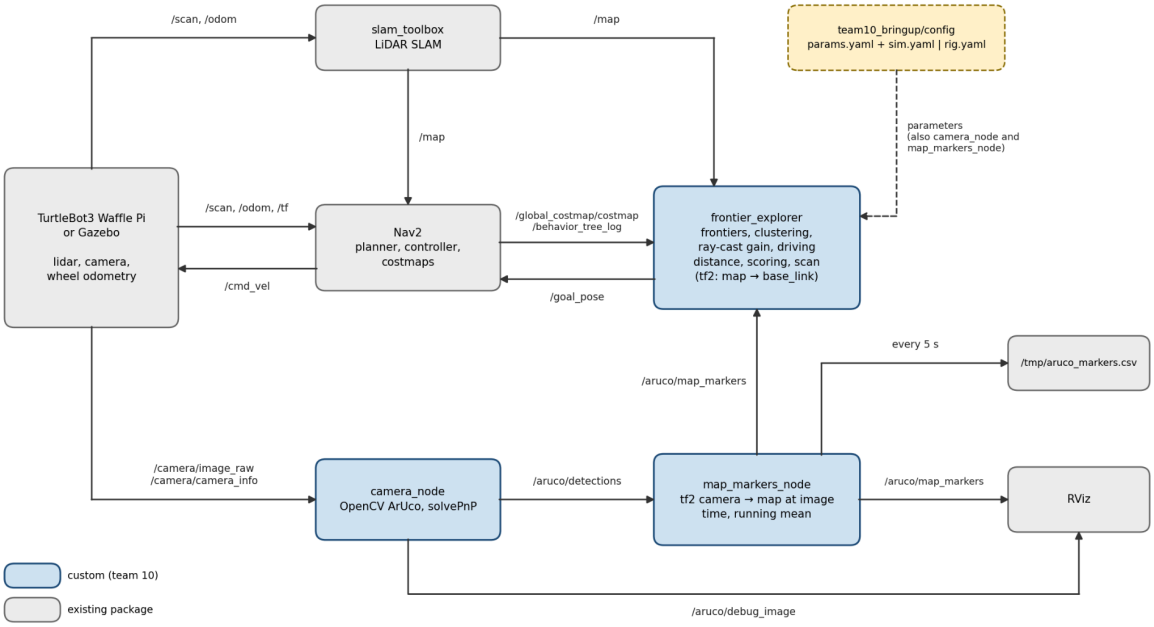


Figure 1: System architecture diagram showing comparison of existing nodes against custom nodes.

Packages and Nodes

Package	Nodes or launch files	Role in the system	Custom or integrated
exploration	frontier_explorer (explorer_node.py, frontier_detection.py, waypoint_finder.py, path_distance.py, waypoint_decider.py), exploration_only.launch.py	Decides where the robot moves next. Reads SLAM and Nav2 maps, finds frontier cells, clusters them, scores them by visible information gain and driving distance, publishes the best as a goal. Also visits mapped markers and handles goal management.	Custom
mapping_pkg	camera_node, map_markers_node, camera_only.launch.py	Finds and maps ArUco targets. camera_node runs ArUco detector on each camera frame, reports relative	Custom

Package	Nodes or launch files	Role in the system	Custom or integrated
		marker position. map_markers_node uses tf2 to transform detections into map frame, keeping a running average per marker with outlier rejection.	
mapping_interfaces	MarkerDetection.msg, MarkerDetectionArray .msg	Defines message between two camera nodes - MarkerDetection carries marker ID, position, range, pixel area. MarkerDetectionArray wraps a timestamped batch of individual detections.	Custom
team10_bringup	stack.launch.py, full_system.launch.py, team10.launch.py, config/params.yaml, sim.yaml, rig.yaml	Starts the system in the right order and holds every tuning value. Brings up Nav2 with SLAM, camera nodes and explorer node, or only the team's nodes on top of a running Nav2.	Custom
metr4202_test_worlds	horseshoe, marker_gauntlet, slam_stress launch files and worlds	Three custom generated Gazebo arenas for testing certain edge cases: difficult marker detection, difficult arena mapping and difficult arena exploration.	Custom
turtlebot3_navigation2 (external)	navigation2.launch.py with slam:=True	Launch entry point for navigation with slam_toolbox.	Integrated
slam_toolbox		Builds an occupancy	Integrated

Package	Nodes or launch files	Role in the system	Custom or integrated
(external)		grid from LiDAR scans and publishes the map-to-odom transform.	
Nav2 (external)		Executes each waypoint, maintaining global and local costmaps and planning a collision-free path, driving robot.	Integrated

Data Flow and Topics

Topic	Message type	Publisher	Subscriber(s)
/scan	sensor_msgs/LaserScan	Robot or Gazebo	slam_toolbox, Nav2
/map	nav_msgs/OccupancyGrid	slam_toolbox	frontier_explorer, Nav2
/global_costmap/costmap	nav_msgs/OccupancyGrid	Nav2	frontier_explorer
/goal_pose	geometry_msgs/PoseStamped	frontier_explorer	Nav2
/behavior_tree_log	nav2_msgs/BehaviorTreeLog	Nav2	frontier_explorer
/cmd_vel	geometry_msgs/Twist	Nav2, frontier_explorer (scan at each goal)	Robot or Gazebo
/camera/image_raw (or /compressed)	sensor_msgs/Image or CompressedImage	Robot or Gazebo	camera_node
/camera/camera_info	sensor_msgs/CameraInfo	Robot or Gazebo	camera_node
/aruco/detections	mapping_interfaces/MarkerDetectionArray	camera_node	map_markers_node
/aruco/map_markers	visualization_msgs/MarkerArray	map_markers_node	frontier_explorer, RViz

Topic	Message type	Publisher	Subscriber(s)
/aruco/debug_image	sensor_msgs/Image	camera_node	RViz, rqt_image_view
/tf, /tf_static	tf2_msgs/TFMessage	slam_toolbox, robot_state_publisher	frontier_explorer, map_markers_node, Nav2

Launch Sequence and Runtime Configuration

There are three launch entry points in team10_bringup. stack.launch.py is used to start everything but the simulator (Nav2 with SLAM, camera nodes, frontier explorer) and runs on any already running Gazebo simulator or the physical TurtleBot. full_system.launch.py is a convenience wrapper to launch a test world alongside the rest of the stack. team10.launch.py starts only the camera nodes and the explorer when SLAM and Nav2 are already running, which is how the real robot is run. Nav2 (with slam set to true) and the camera nodes are launched initially, and the explorer is launched after 10 seconds to allow slam_toolbox to publish the first map, and for Nav2's global costmap and action servers to come online.
Every tuning value for the three custom nodes is a ROS parameter in team10_bringup/config/params.yaml. A second file, sim.yaml or rig.yaml, is loaded on top of it with the few values that differ between Gazebo and the real robot (camera topic, compressed flag, marker size, camera frame), chosen with the config launch argument. Edits take effect on the next launch without a rebuild, and frontier_explorer and map_markers_node re-read their values while running, so a weight can be changed with ros2 param set and used from the next goal.

Developed Components

Frontier Detection

Frontier detection takes the occupancy data from OccupancyGrid and returns a list of frontier cells. Frontier cells are free cells with at least one unknown neighbour among the four edge-adjacent cells. The SLAM map encodes each cell as 0 for free, 100 for occupied, and -1 for unknown. The detector only treats cells with a score of 0 as free. The scan visits every cell once and checks at most four neighbours, making the cost linear in the number of map cells.

Frontier Clustering and Waypoint Candidates

Frontier cells are clustered by a breadth-first search over their eight neighbours. Each cluster is capped at half of the lidar range (1.75 m), which splits longer frontiers into several waypoints. Any cluster smaller than 5 cells is discarded. The waypoint is calculated as the cell nearest to the cluster centroid. Once the waypoint is in map-frame coordinates, the explorer algorithm also moves it to the nearest cell in the Nav2 global costmap with a cost of 80 or less so it sits outside the inflation ring around the walls.

Information Gain Estimate

Information gain is the unknown area the lidar could see from a waypoint. 120 rays are cast from the waypoint cell out to 2.5 m, each stopping at the first occupied cell, and the unknown cells they cross are summed as an area in square metres. Unknown space behind a wall, such as outside the arena, therefore does not count. This replaced a simple count of unknown cells within a radius, which made the robot favour the corners of the arena.

Driving Distance

The distance used in scoring is the distance the robot would have to drive, not the straight-line distance through walls. One Dijkstra search over the free cells of the SLAM map, starting from the robot's cell, gives the driving distance to every free cell at once. Steps go to the eight neighbours and a diagonal step is not allowed to cut between two cells that are not free. A waypoint the robot cannot reach gets no distance and is never sent, which removes goals seen through gaps in the lidar scan.

Waypoint Scoring and Exploration Strategy

Waypoint scoring picks one goal from the candidate waypoints. Each waypoint is given a utility score $U = a \times (\text{gain} + \text{bonus}) - b \times \text{distance}$, where gain is the visible unknown area around the waypoint, bonus is 50 m^2 for a marker view pose and 0 otherwise, and distance is the driving distance from the robot to it. The weights are $a = 8$ and $b = 1$. Waypoints within 0.5 m of the robot, within 0.5 m of a previously failed goal, or unreachable are skipped. The best waypoint within 3 m wins unless a further one offers 1.5 times as much gain, so the robot finishes the area it is in before crossing the map. The strategy uses only the live map and robot pose, so it

works in any world without tuning for a specific environment.

Marker Visiting

The explorer also subscribes to the mapped markers on /aruco/map_markers. Each marker not yet visited becomes a candidate waypoint: a view pose within 1 m of the marker, on the robot's side of it, in a costmap cell with cost 30 or less, with the goal facing the marker. The marker bonus ranks these view poses above most frontiers, so the robot inspects each marker at close range soon after it is first seen. A marker counts as visited once the robot has reached its view pose or given up on it. Exploration is complete when there are no frontier waypoints and no unvisited markers.

Explorer Node and Nav2 Goal Handling

The explorer node handles the entire exploration pipeline. It holds all of the states about the latest map, costmap, mapped markers, current goal and navigation, running frontier detection, clustering, gain, driving distance and scoring, publishing the best waypoint on /goal_pose and watching the behaviour tree log to flag when each goal is reached. At a goal it reached, the robot sits still for 2 s and then scans 30° to the left and right at 0.4 rad/s so the front camera covers a wider view, unless the costmap shows it is too close to a wall to turn on the spot. The goal and behaviour log pattern follows the Prac 4 waypoint cycler.

Failure Detection and Recovery

Failure detection occurs in three cases. A goal can be blacklisted if Nav2 completes navigation while the robot is further than 0.5 m from the goal, indicating goals aborted or not planned by Nav2. Secondly a goal is blacklisted if it is still active after 60 seconds, indicative of a stuck or oscillating robot. Thirdly, any goal within 0.5 m of a blacklisted goal is skipped in scoring, and goals with no driving path are skipped before scoring. If every waypoint ends up blacklisted the list is cleared and scoring runs again. A marker view pose that fails or times out marks the marker as visited so it is not retried forever.

ArUco Marker Detection

The camera node runs OpenCV's ArUco detector on each camera frame using the DICT_6X6_1000 dictionary with sub-pixel corner refinement, and converts compressed or raw images with cv_bridge. For each marker found it estimates the marker's position relative to the camera with solvePnP, using the 100 mm marker size and the intrinsics from /camera/camera_info. If the images arrive at a different resolution than the camera was calibrated at, the intrinsics are scaled to the image size, and the detections are stamped with the camera's optical frame since Gazebo stamps its images with the camera body frame. The node publishes each marker's ID, camera-frame position, range and pixel area on /aruco/detections, and an annotated debug image for RViz.

Marker Localisation in the Map Frame

The map markers node receives each detection batch and uses tf2 to look up the camera pose at the image timestamp, transforming each marker position into the map frame. Detections further than 2.3 m from the camera are discarded, and each marker's map position is kept as a running mean of its sightings, with any sighting more than 1 m from the mean ignored once a marker has been seen five times. The node publishes the current estimates as a MarkerArray for RViz and the explorer, and writes them to a CSV file every five seconds.

Custom Message Interfaces

The mapping_interfaces package defines two messages for passing detections between the camera node and the map markers node. MarkerDetection holds one marker's ID, its position in the camera frame, its range in metres and its pixel area in the image. MarkerDetectionArray wraps a list of these with a standard header so the map markers node can look up the camera pose at the exact time the image was taken.

System Bringup

The team10_bringup package holds the launch files and configuration that start the system. stack.launch.py starts Nav2 with SLAM through the TurtleBot3 navigation launch file, the two camera nodes, and the explorer after a 10 s delay so a first map exists before frontiers are searched. full_system.launch.py launches one of the team's Gazebo worlds, waits 10 s for the robot to spawn, then includes stack.launch.py. team10.launch.py starts only the camera nodes and the explorer on top of a running Nav2, with sim time off and the rig configuration by default for the real robot.

Test Worlds

The metr4202_test_worlds package provides three extra Gazebo arenas built to the same conventions as the course reference world: a 10 m by 10 m arena, 0.5 m walls, and 100 mm ArUco tags on wooden posts. Each world targets a failure mode the reference maze does not exercise: horseshoe has targets that are close in a straight line but far by road, marker_gauntlet has tags that are high, recessed, tilted or occluded, and slam_stress has featureless corridors, repeated bays and thin walls. Each world has a launch file that spawns the Waffle Pi, a variant without ArUco columns, and a ground-truth file of marker positions for checking results. The worlds were generated with the help of an AI model.

Use of Existing Code and Libraries

The exploration pipeline, marker detection and mapping nodes, messages, launch files, configuration and test worlds were written by the team. Everything below was used as installed.

Library or existing code	Version	Where it is used	Modifications made	Reference
ROS 2 Humble (rclpy, launch, tf2_ros, tf2_geometry_msgs)	Humble Hawksbill	All nodes and launch files. tf2 gives the explorer its robot pose and transforms marker detections into the map frame.	None	[1]
Nav2 (navigation2, nav2_bringup)	Humble apt (1.1.x)	Path planning, costmaps and velocity control for every goal. The explorer reads the global costmap and behaviour tree log.	None. Stock Waffle Pi parameters.	[2]
slam_toolbox	Humble apt (2.6.x)	Builds the occupancy grid from lidar and publishes the map to odom transform.	None	[3]
TurtleBot3 packages (turtlebot3, turtlebot3_simulations, turtlebot3_navigation2)	Humble apt (2.1.x / 2.2.x)	Robot model, Gazebo spawn, and the navigation2 launch file that stack.launch.py includes.	None	[4]
Gazebo Classic and gazebo_ros_pkgs	Gazebo 11	Simulation of the robot and all test worlds.	None	[5]

Library or existing code	Version	Where it is used	Modifications made	Reference
OpenCV (python3-opencv, cv2.aruco)	4.5.x	Marker detection, corner refinement and solvePnP pose estimation in camera_node.	None	[6], [7]
cv_bridge	Humble apt (3.2.x)	Converts ROS Image and CompressedImage messages to OpenCV arrays in camera_node.	None	[8]
NumPy	1.21.x	Array maths in the detection pipeline and marker averaging.	None	[9]
ROS 2 tf2 tutorials	Humble docs	Structure of the tf2 listener in map_markers_node.	Adapted to a PointStamped lookup at the image time.	[10]
metr4202_aruco_explore (course reference world)	Course repository	Reference demo world used for testing. Its arena, wall and marker post conventions were followed when building the team's worlds.	None. The team worlds are new files in a separate package.	[11]
Prac 4 waypoint_cycler (course material)	Semester 2, 2026	The pattern of publishing a PoseStamped to /goal_pose and reading /behavior_tree_log in the explorer node.	Fixed waypoint list replaced by the frontier pipeline. Hold, scan, blacklist and timeout added.	[12]

Interim Assessment Checklist Compliance

Checklist item	Component(s) responsible	Report section	Evidence
Waypoints are automatically computed and sent to the robot	frontier_explorer node (waypoint_finder, path_distance, waypoint_decider), Nav2	Waypoint Scoring; Explorer Node	Explorer logs a chosen goal line for each waypoint. Goals appear in RViz without user input (Figures 2 to 5).
SLAM map generated from LiDAR measurements	slam_toolbox via turtlebot3_navigation 2 launch	System Overview	/map topic in RViz during the demo. Final maps in Figures 2 and 4.
Algorithm for detecting unexplored areas	frontier_detection.py	Frontier Detection	Unit tests in test_exploration_logic .py. Goals are sent to the boundary of the mapped area in RViz.
Documented exploration strategy	waypoint_finder.py, path_distance.py, waypoint_decider.py	Waypoint Scoring and Exploration Strategy	This report and the Exploration strategy section of the README.
Strategy works in all world files (not hard-coded)	frontier_explorer node	Testing and Results	Runs in the reference world and the three team worlds with no code or parameter changes. Only the live map, costmap and robot pose are used.
Mechanism for detecting when exploration strategy fails	frontier_explorer node (unreachable skip, blacklist, goal timeout, completion check)	Failure Detection and Recovery	Explorer logs goal failed, blacklisted and timed out warnings, and no frontiers or markers left,

Checklist item	Component(s) responsible	Report section	Evidence
			exploration complete when finished.
Navigation to unexplored areas provides collision-free motion	Nav2 costmaps and planner, goal snapping and driving-distance check in frontier_explorer	Failure Detection and Recovery; Frontier Clustering; Driving Distance	Goals are only sent to cells with costmap cost of 80 or less that are reachable over free cells. Collision-free runs in the demo and team worlds (Figures 3 and 5).

Testing and Results

Test Environments

- Testing was done in Gazebo in four arenas: the course reference maze from metr4202_aruco_explore and the three team worlds, horseshoe, marker_gauntlet and slam_stress.
- Each team world isolates one failure mode: path length versus straight-line distance, hard-to-read markers, and SLAM drift in featureless corridors.
- The same launch files and parameters were used in every world, with only the world argument changed.
- The stack was also run on the physical TurtleBot3 with SLAM and Nav2 started separately and team10.launch.py on top, using the rig configuration.

Unit Tests

The exploration package has a pytest suite, test_exploration_logic.py, that tests the logic modules without ROS running. Small occupancy grids are built from strings of free, unknown and occupied cells and passed through the same functions the explorer node calls. The tests are grouped by module: frontier detection, clustering and waypoint selection, information gain, driving distance, and the decider's scoring, skipping and local-first rules. Each group checks the normal case and the edge cases the module was written for, such as a fully known map, a frontier split by the size cap, unknown space behind a wall, and a goal with no path. All tests pass and are run with python3 -m pytest.

Simulation Results

- Exploration covers the whole arena. Goals are sent automatically to the edge of the mapped area until the map is complete, as in Figures 2 and 4, where only a single unexplored region remains at the edge of the map.
- Goals are reached. The Nav2 panel in Figures 5 and 6 shows goals reached with no recoveries, and in Figure 3 the planned path to a goal behind a wall follows the corridor around it rather than the straight line.
- Markers are placed on the map. The green markers in Figures 4 to 7 sit on the marker posts, the camera image in Figure 6 shows the tag in view at the view pose, and four markers were mapped in the marker_gauntlet run in Figure 7.

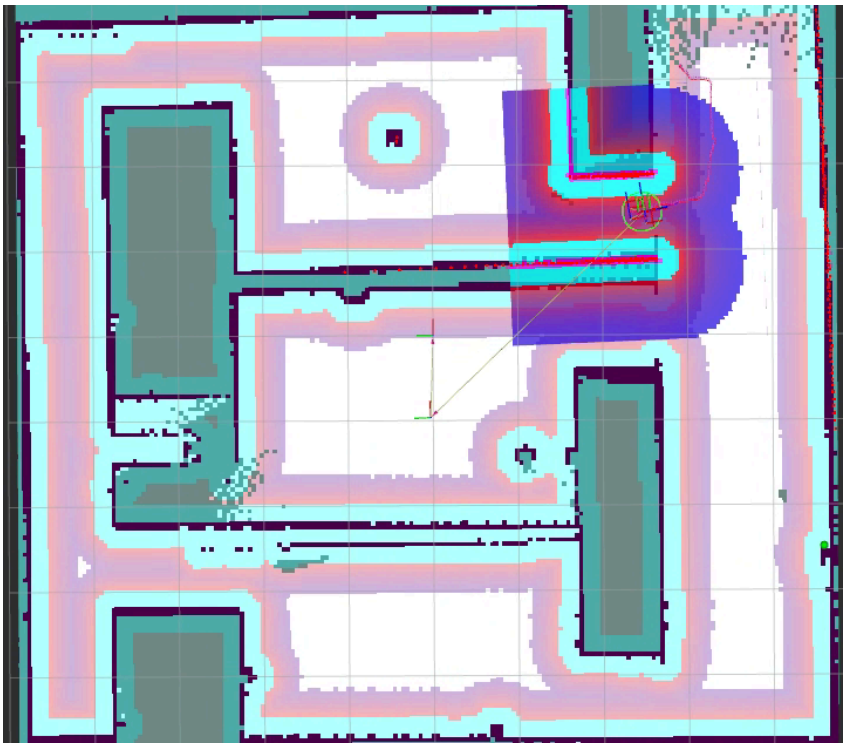


Figure 2: Exploration near completion, with the robot clearing the last unexplored region of the arena.

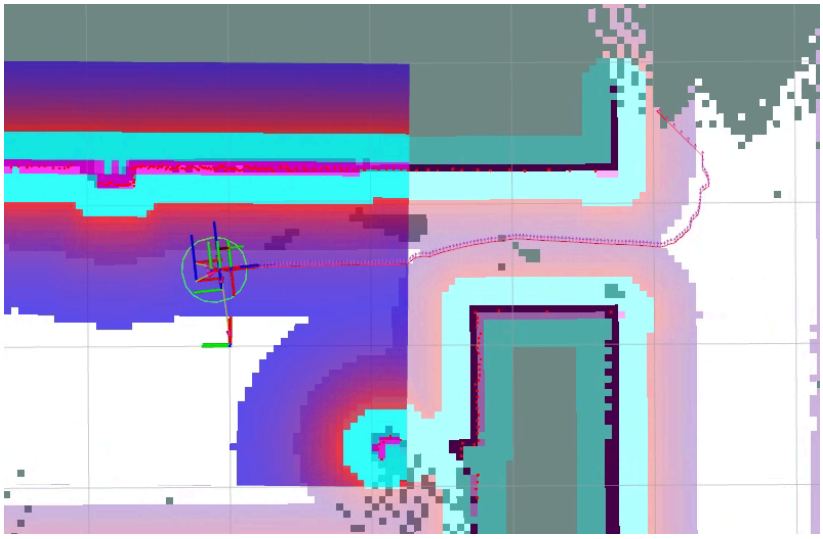


Figure 3: A goal sent to a frontier on the far side of a wall. The planned path follows the corridor around the wall.

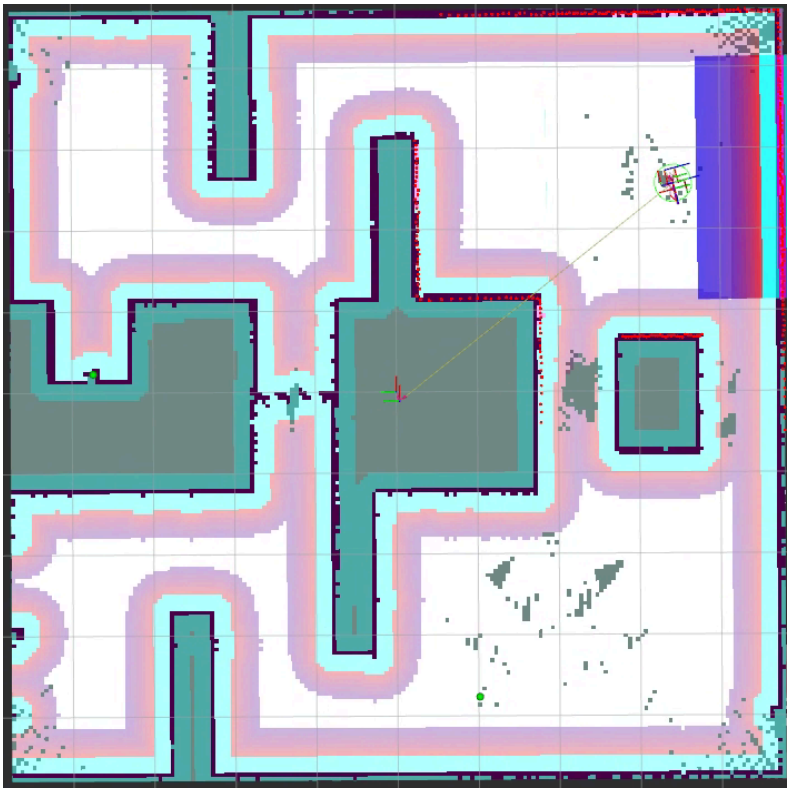


Figure 4: Fully explored arena with a mapped marker (green) and the final frontier at the right edge.

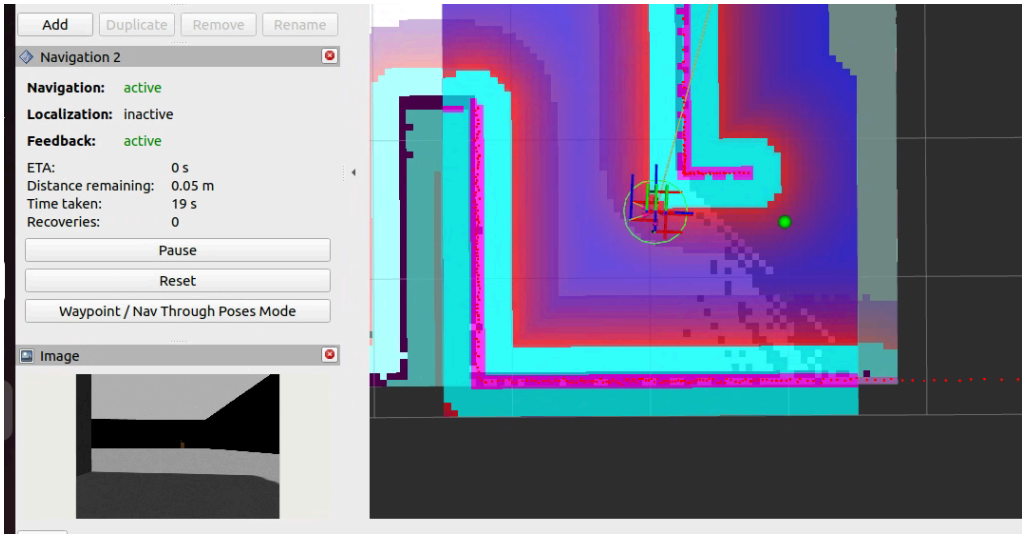


Figure 5: Nav2 panel while reaching a goal beside a mapped marker: 0.05 m remaining after 19 s with no recoveries.

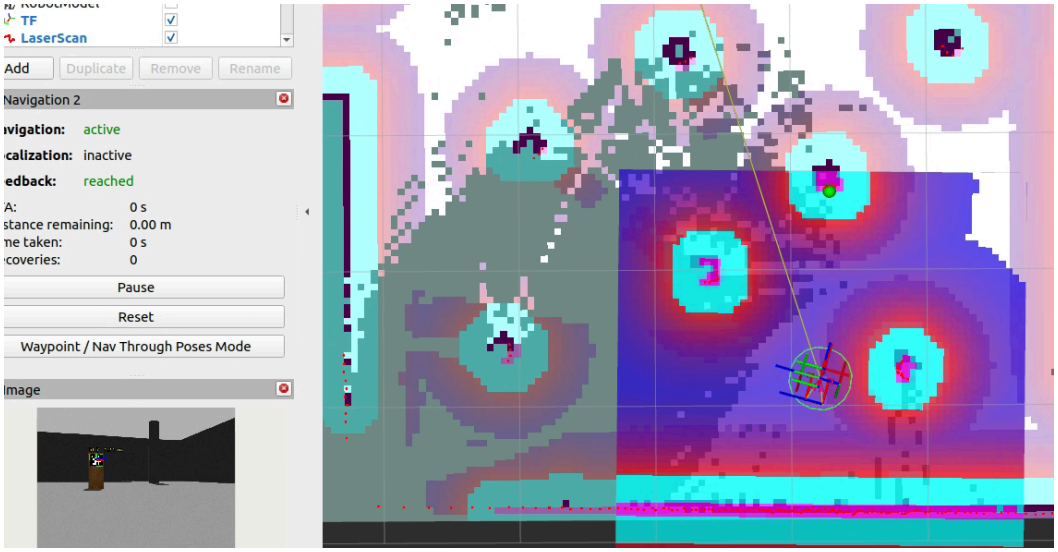


Figure 6: marker_gauntlet. The robot at a marker view pose with the tag visible in the camera image and the marker placed on its post.

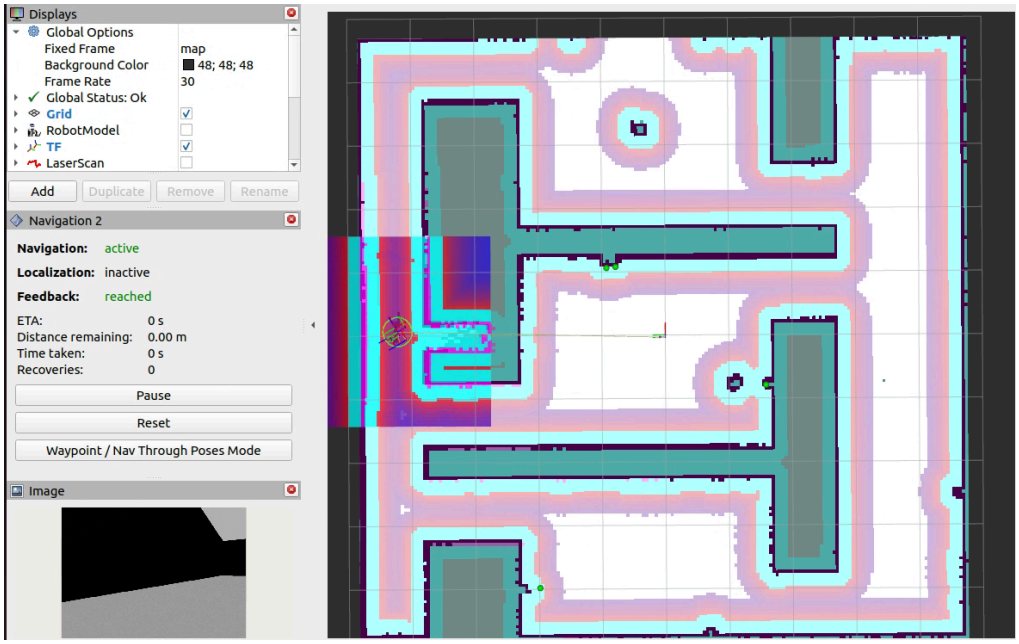


Figure 7: Exploration of marker_gauntlet with four identified markers mapped onto RViz.

Real Robot Results

- The full stack ran end to end on the physical TurtleBot3: SLAM map, Nav2 goals from the explorer, and ArUco markers placed on the map.
- With the original straight-line scoring and radius gain the robot favoured corners of the arena and moved back and forth between two of them. This was traced to unknown space outside the arena walls being counted as gain, and was fixed by the ray-cast gain and driving distance.
- Markers were initially placed 1 to 1.5 m beyond the wall they were on. The camera streams at a different resolution than its calibration, so the intrinsics are now rescaled and the maximum range was set to 2.3 m.

Observed Failures and Limitations

- Marker detections are missed while the robot turns quickly because of motion blur, so the scan at each goal is done slowly.
- The marker position is a running mean, so one early bad sighting shifts the estimate until later sightings pull it back.
- If every candidate is blacklisted the list is cleared and the same goals can be retried, which can loop at the end of a run rather than finishing cleanly.
- The physical marker size has not yet been measured on the rig, so ranges on the real robot may carry a scale error.

Individual Contributions

Team member	Components owned	Description of technical contribution	Approximate share
Mitch Bradshaw	Mapping packages (camera_node.py, aruco_core.py),path_distance.py,	Main responsibility for implementing the algorithm to detect ArUco markers and send relative locations to mapping algorithms. Implemented the ray-cast information gain and driving-distance cost,	0.25
Brad Cronin	Mapping packages (map_markers.py)	Main responsibility to map relative marker location to map location using tf2 transforms, the running mean and outlier rejection, and the RViz and CSV outputs.	0.25
Einar Augestad	Exploration node (explorer_node.py), team10.launch.py	Main responsibility for implementing the exploration node that runs the autonomous exploration loop, goal handling and failure detection. Added the team10 launch file for the real robot. Also helped integrate the waypoint decision pipeline.	0.25
Ingrid Skålnes Strømme	Frontier detection, finding best waypoint (frontier_detection.py, waypoint_finder.py, waypoint_decider.py), marker goals	Had main responsibility for the frontier detection and waypoint selection. Developed the algorithm to identify frontiers in the SLAM map, implemented clustering of frontier cells, and developed a function to calculate waypoints within clusters. Also implemented the estimation of exploration gain and distance for each waypoint, and selection of waypoint based on gain and distance, and added the logic that turns mapped markers into prioritised waypoints. Also	0.25

Team member	Components owned	Description of technical contribution	Approximate share
		helped with implementation of the exploration node.	

Installation and Operation

Full instructions are in the repository README. In summary: install the ROS 2 Humble packages for Nav2, slam_toolbox, TurtleBot3, Gazebo, cv_bridge and OpenCV with apt, clone this repository and the course world package into the src folder of a workspace, set TURTLEBOT3_MODEL to waffle_pi and build from the workspace root with colcon using a symlink install. In simulation, launch the reference world in one terminal and stack.launch.py in another, or run full_system.launch.py with a world argument for a team world. On the real robot, bring up the TurtleBot, SLAM and Nav2 as normal, check the camera topic matches rig.yaml, and run team10.launch.py. After about 20 s the explorer prints that it has started and then, for every goal, the top candidates and a chosen goal line. In RViz, add a MarkerArray display on /aruco/map_markers and an Image display on /aruco/debug_image. The unit tests run with pytest on the exploration test file.

References

1. Open Robotics, "ROS 2 Documentation: Humble", https://docs.ros.org/en/humble, accessed October 2026.
2. S. Macenski, F. Martín, R. White and J. Ginés Clavero, "The Marathon 2: A Navigation System", IEEE/RSJ International Conference on Intelligent Robots and Systems, 2020. Nav2 documentation, https://docs.nav2.org.
3. S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world", Journal of Open Source Software, vol. 6, no. 61, 2021. https://github.com/SteveMacenski/slam_toolbox.
4. ROBOTIS, "TurtleBot3 e-Manual", https://emanual.robotis.com/docs/en/platform/turtlebot3/overview, accessed October 2026.
5. Open Source Robotics Foundation, "Gazebo Classic 11", https://classic.gazebosim.org, accessed October 2026.
6. OpenCV, "Detection of ArUco Markers", https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html, accessed October 2026.
7. S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion", Pattern Recognition, vol. 47, no. 6, pp. 2280-2292, 2014.
8. ROS Perception, "vision_opencv: cv_bridge", https://github.com/ros-perception/vision_opencv, accessed October 2026.
9. C. R. Harris et al., "Array programming with NumPy", Nature, vol. 585, pp. 357-362, 2020.
10. Open Robotics, "Writing a tf2 listener (Python)" and "Adding a frame (Python)", ROS 2 Humble tutorials, https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2, accessed October 2026.
11. METR4202, "metr4202_aruco_explore", https://github.com/METR4202/metr4202_aruco_explore, accessed October 2026.
12. METR4202, "Prac 4: Navigation with Nav2", The University of Queensland, Semester 2, 2026.
13. B. Yamauchi, "A frontier-based approach for autonomous exploration", IEEE International Symposium on Computational Intelligence in Robotics and Automation, pp. 146-151, 1997.
14. E. W. Dijkstra, "A note on two problems in connexion with graphs", Numerische Mathematik, vol. 1, pp. 269-271, 1959.

Appendices

Appendix A: Repository Structure

```
metr4202_2026_team10/              workspace src folder
├── README.md                      installation and operation instructions
├── exploration/                   frontier exploration (ament_python)
│   ├── exploration/
│   │   ├── explorer_node.py       the frontier_explorer ROS node
│   │   ├── frontier_detection.py  frontier cell detection
│   │   ├── waypoint_finder.py     clustering, waypoints, ray-cast gain
│   │   ├── path_distance.py       Dijkstra driving distance over the map
│   │   └── waypoint_decider.py    utility scoring and local-first rule
│   ├── launch/exploration_only.launch.py
│   └── test/test_exploration_logic.py   pytest unit tests
├── mapping_pkg/                   ArUco detection and mapping (ament_python)
│   ├── mapping_pkg/
│   │   ├── aruco_core.py          OpenCV detection and pose estimation
│   │   ├── camera_node.py         camera_node ROS node
│   │   └── map_markers.py         map_markers_node ROS node
│   └── launch/camera_only.launch.py
├── mapping_interfaces/            custom messages (ament_cmake)
│   └── msg/
│       ├── MarkerDetection.msg
│       └── MarkerDetectionArray.msg
├── team10_bringup/                launch files and configuration (ament_python)
│   ├── config/
│   │   ├── params.yaml            every tuning value for the three nodes
│   │   ├── sim.yaml               Gazebo overrides (camera topic, marker size, camera
frame)
│   │   └── rig.yaml               real robot overrides
│   └── launch/
│       ├── stack.launch.py        Nav2 + SLAM, camera nodes, explorer
│       ├── full_system.launch.py  team world + stack
│       └── team10.launch.py       camera nodes + explorer only
└── metr4202_test_worlds/          extra Gazebo worlds (ament_cmake)
    ├── README.md
    ├── launch/                    horseshoe, marker_gauntlet, slam_stress
    ├── worlds/                    SDF files, with and without ArUco columns
    ├── docs/                      world images and marker ground truth
    ├── resource/media/            ArUco textures and material script
    └── tools/generate_worlds.py   world generator
```

Appendix B: Tunable Parameters

All values are in team10_bringup/config/params.yaml unless noted. frontier_explorer and map_markers_node re-read theirs each cycle, so they can be changed live with ros2 param set.

Parameter	Node	Default	Effect
a	frontier_explorer	8.0	Weight on information gain and marker bonus, per square metre.
b	frontier_explorer	1.0	Weight on driving distance, per metre.
sensor_range_m	frontier_explorer	3.5 m	Lidar range. Frontier clusters are capped at half of it.
gain_radius_m	frontier_explorer	2.5 m	How far the gain rays reach. Walls stop them.
min_cluster_size	frontier_explorer	5 cells	Smaller frontier clusters are ignored as noise.
max_goal_cost	frontier_explorer	80	Highest costmap value a goal may sit on.
nudge_radius_m	frontier_explorer	0.5 m	How far a goal may be moved to find a low-cost cell.
min_travel_dist	frontier_explorer	0.5 m	Candidates this close to the robot are skipped.
blacklist_radius	frontier_explorer	0.5 m	Candidates this close to a failed goal are skipped.
goal_reached_dist	frontier_explorer	0.5 m	A goal finished further away than this is blacklisted.
goal_timeout_s	frontier_explorer	60 s	A goal active longer than this is blacklisted.

Parameter	Node	Default	Effect
hold_s	frontier_explorer	2 s	Pause after each goal so the camera gets frames.
scan_angle_deg, scan_speed, scan_max_cost	frontier_explorer	30°, 0.4 rad/s, 90	Left-right scan at each reached goal. Skipped if the costmap cost under the robot is above scan_max_cost. scan_speed 0 turns it off.
marker_bonus	frontier_explorer	50 m²	Gain a marker view pose counts as.
marker_view_radius_m	frontier_explorer	1.0 m	How far from a marker its view pose may be.
marker_view_max_cost	frontier_explorer	30	Costmap cost a view pose must be under.
local_radius_m	frontier_explorer	3.0 m	Candidates within this driving distance are finished first. 0 turns it off.
far_ratio	frontier_explorer	1.5	A further candidate wins only if it offers this many times more.
image_qos_depth	camera_node	5	Image queue length. 1 always processes the newest frame.
aruco_dictionary	camera_node	DICT_6X6_1000	Marker dictionary used by the detector.
marker_size (sim.yaml, rig.yaml)	camera_node	0.1 m	Black square side length used for pose estimation.
image_topic, use_compressed (sim.yaml, rig.yaml)	camera_node	/camera/image_raw, false (sim); /camera/image_raw/compressed, true (rig)	Camera stream to subscribe to.

Parameter	Node	Default	Effect
camera_frame (sim.yaml)	camera_node	camera_rgb_optical_frame (sim); unset (rig)	Frame the detections are stamped with when the camera driver uses a body frame.
map_frame	map_markers_node	map	Frame markers are placed in. Use odom to test without SLAM.
max_range_m	map_markers_node	2.3 m	Sightings further away are discarded.
outlier_min_sightings, outlier_dist_m	map_markers_node	5, 1.0 m	After five sightings, sightings further than 1 m from the mean are ignored.
output_file	map_markers_node	/tmp/aruco_markers.csv	Marker positions written here every 5 s.

Appendix C: Launch Arguments

Argument	Launch file	Default	Description
world	full_system.launch.py	marker_gauntlet	Team world to load: horseshoe, marker_gauntlet or slam_stress.
gui	full_system.launch.py, world launch files	true	false runs Gazebo headless.
use_sim_time	stack, full_system, camera_only, exploration_only	true	Use the Gazebo clock.
use_sim_time	team10.launch.py	false	Set for the real robot.
config	stack, full_system, camera_only, exploration_only	sim	Which override file to load after params.yaml: sim or rig.
config	team10.launch.py	rig	Set for the real robot.
aruco	world launch files	true	false loads the same maze without ArUco columns.
x_pose, y_pose, yaw	world launch files	per world	Robot spawn pose.
odometry	world launch files	world	world gives ground-truth odom; encoder gives drifting wheel odometry.
odom_error	world launch files	0.0	Fractional wheel calibration error when odometry is encoder.

Appendix D: Acknowledgement of AI Use

An AI model (Claude, Anthropic) was used during this project. It helped generate the three extra Gazebo test worlds in metr4202_test_worlds, gave advice on exploration algorithms and mapping algorithms and corrected grammar in the report text. All code in the exploration and mapping packages was written, tested and integrated by the team, and all design decisions were made by the team.